SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes
QUESTIONS: 2 **Total Marks:20**

Annexure A **INTEGRATED**

EXPERIMENTS

Code of Conduct:

I V.SURYA SATVIKA(192524254)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V.S.SATVIKA

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / PlayTennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

1(a) Dataset Loading and Preprocessing

The dataset can be loaded using Python and a suitable machine learning library. For this experiment, the Iris dataset can be used.

The dataset is first inspected to identify missing values and understand the data types. If missing values are present, they are handled using suitable methods such as mean or median imputation. Categorical data, if present, is converted into numerical form using encoding.

The dataset is then divided into features (X) and the target variable (y). Finally, the dataset is divided into training and testing sets. The training set is used to build the Decision Tree, while the testing set is used to evaluate its performance.

Steps:

1. Load the dataset.
2. Display the first five rows.
3. Check the data types of all columns.
4. Check and handle missing values.
5. Encode categorical variables if required.
6. Separate features and target.

7. Split the dataset into training and testing sets.

1(b) Decision Tree Model

A Decision Tree is a supervised machine learning algorithm used for classification. It divides the dataset into different branches based on feature conditions.

For this experiment, the Decision Tree can be constructed using either the Gini Index or Information Gain as the splitting criterion.

The root node is selected based on the feature that provides the best separation of the target classes.

Gini Index:

$Gini = 1 - \Sigma p_i^2$

The Gini Index measures the impurity of a node. A feature producing the lowest Gini impurity is preferred for splitting.

Information Gain:

Information Gain = Entropy(parent) − Weighted Entropy(children)

Information Gain measures the reduction in entropy after a split. The feature with the highest Information Gain is selected as the splitting feature.

The tree structure is then printed to show the root node, internal nodes and leaf nodes.

Therefore, the root node is the feature that gives the best separation of the target classes according to the selected splitting criterion.

1(c) Decision Tree Evaluation

After training the Decision Tree, the model is evaluated using the testing dataset.

1. Accuracy Score

Accuracy measures the percentage of correctly classified instances.

Accuracy = Correct Predictions / Total Predictions

A higher accuracy indicates better classification performance.

2. Confusion Matrix

The confusion matrix shows the number of correctly and incorrectly classified samples for each class.

It contains True Positives, True Negatives, False Positives and False Negatives.

3. Classification Report

The classification report provides Precision, Recall, F1-score and Support.
The model performance can be considered good if it achieves high accuracy, precision, recall and F1-score.

Misclassified Instances:

Misclassified instances are samples for which the predicted class is different from the actual class. At least two such instances can be identified by comparing the actual and predicted labels from the test dataset. Therefore, the confusion matrix and classification report help in understanding the strengths and weaknesses of the Decision Tree classifier.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

2(a) Q-Learning Environment

Q-Learning is a model-free reinforcement learning algorithm that learns the best action for each state by interacting with an environment.

For this experiment, a 4×4 Grid World is used.

States:

The environment contains 16 states, corresponding to the 16 cells of the 4×4 grid.

Actions:

The agent can perform four actions:

- Up
- Down
- Left
- Right

Reward Structure:

- +10 for reaching the Goal.
- −1 for each step.
- −5 for hitting an obstacle.

The positive reward encourages the agent to reach the goal, while negative rewards encourage the agent to avoid unnecessary steps and obstacles.

Q-Table:

The Q-table stores the expected reward for each state-action pair. Initially, all Q-values are set to zero.

Hyperparameters:

- α (Learning Rate) = 0.1
- γ (Discount Factor) = 0.9
- ε (Exploration Rate) = 0.1

These parameters control the learning rate, importance of future rewards and exploration of actions.

2(b) Q-Learning Episodes and Q-Table

The Q-Learning algorithm is executed for at least 100 episodes. During each episode, the agent starts from an initial state and selects actions using an ε-greedy strategy.

The Q-value is updated using:

$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)]$

The Q-table is initially filled with zeros. As the agent interacts with the environment, the Q-values are updated according to the rewards received.

Example Q-Value Progress:

Episode	State	Up	Down	Left	Right
1	State 0	0.00	0.00	0.00	0.00

Episode	State	Up	Down	Left	Right
1	State 5	0.00	0.00	0.00	0.00
50	State 0	0.00	2.41	0.00	5.72
50	State 5	3.16	0.00	1.52	6.48
100	State 0	0.00	5.21	0.00	7.84
100	State 5	4.62	0.00	2.31	8.25

The values shown are an illustrative example. Actual Q-values depend on the implementation, environment and random exploration.

As the number of episodes increases, the Q-values for useful actions generally increase, while actions leading to poor rewards receive lower values. Thus, the Q-table gradually learns the optimal actions for different states.

2(c) Final Learned Policy and Convergence

After completing 100 or more episodes, the final policy is obtained from the Q-table.

For each state, the agent selects the action having the highest Q-value.

Policy(s) = argmax_a Q(s,a)

The final policy can be represented using arrows:

- ↑ = Up
- ↓ = Down
- ← = Left
- → = Right

Example Policy:

→ → ↓ ↓
↑ → → ↓
↑ ↑ → ↓
→ → → G

Here, G represents the goal.

Cumulative Reward:

The cumulative reward for each episode is recorded and plotted. Initially, the rewards may be low or negative because the agent explores different actions. As learning progresses, the agent discovers better paths to the goal.

Convergence Behaviour:

The algorithm is considered to show convergence when:

1. The Q-values become relatively stable.
2. The agent repeatedly selects effective actions.
3. The cumulative reward becomes more stable.
4. Further episodes produce only small changes in the Q-table.

Therefore, increasing cumulative rewards and stable Q-values indicate that the agent has learned an effective policy for the Grid World environment.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, wellstructured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation